Building data course — Week 8 — building an ontology on purpose (Ontology Development 101)

sparql-playground / lessons

Generated for print and offline reading

Week 8 — building an ontology on purpose (Ontology Development 101)

Five daily lessons on Burger Barn, a made-up burger restaurant. Each day has one short script, `dayNN_rdf.py`, that reads small hand-written Turtle files next to it in `lessons/unit4-rdf-by-hand/`. Run it with `uv run python lessons/course.py run N`. Every day ends with a box on how it maps to buildings (Brick,

ASHRAE 223P, Project Haystack) and a burger mini challenge; its answer is in the Try it section. Keep notes under the ignored `student-work/` folder.

Contents

- Day 36 — Ontology Development 101: scope, competency questions and reuse
- Day 37 — Steps 4–5: classes, hierarchy and properties
- Day 38 — Steps 6–7: restrictions and instances
- Day 39 — Step 8: test the ontology with its competency questions and SHACL
- Day 40 — Reuse, part two: aligning your terms with Brick

Day 36 — Ontology Development 101: scope, competency questions and reuse

Goal: start a new ontology the way the textbook method does: decide what it's for, write the questions it must answer, look for terms that already exist, and list the terms you still need.

Read

Weeks 6–7 grew Burger Barn's ontology one idea at a time. This week you build one *on purpose*, following the classic beginner's method, Noy and McGuinness's Ontology Development 101. Simplified, it has eight steps:

Step	What you do	This week
1	Set the **scope** and write **competency questions**	today
2	Look for existing ontologies to **reuse**	today (and Day 40)
3	**List the terms** you need	today
4	Define the **classes** and their hierarchy	Day 37
5	Define the **properties**	Day 37
6	Add **restrictions**	Day 38
7	Create **instances**	Day 38
8	**Test**: answer the	Day 39

Step	What you do	This week
	competency questions with SPARQL, validate with SHACL	

Competency questions are the heart of it: plain-English questions the finished ontology must be able to answer, such as “which menu items depend on a fryer?”. They keep the ontology small (no question, no term) and they become its tests on Day 39.

Scope says just as clearly what is *out*: staff, prices and recipes are real, but no question needs them.

Reuse means: before inventing `Equipment`, check whether a standard already has it. Here we look in the few Brick terms in `tiny_building.ttl`. (The real Brick has about a thousand classes, and you’d also check ASHRAE 223P, Haystack, QUDT for units and SKOS for definitions.) `day36_rdf.py` writes the scope and five competency questions, lists every term they mention, and looks for a Brick term with the same name.

Run

```
uv run python lessons/course.py run 36
```

```
Scope: Burger restaurant equipment, its
sensor points, and which menu items need
it.
Out of scope: staff and shifts, prices and
sales, recipes and cooking steps

Competency questions:
   CQ1 Which menu items depend on a fryer?
-> MenuItem, Fryer, cookedOn,
hasIngredient
   CQ2 Which equipment has no temperature
sensor? -> Equipment, hasPoint,
TemperatureSensor
   CQ3 Where is each piece of equipment? -
> Equipment, hasLocation, Kitchen
   CQ4 What oil temperature is each fryer
set to? -> Fryer, oilSetpoint
   CQ5 How many calories is each menu
item? -> MenuItem, hasCalories

Terms needed (11), and a Brick term with
the same name:
   Equipment          brick:Equipment
   Fryer              (none: a new burger:
term)
   Kitchen            (none: a new burger:
term)
   MenuItem           (none: a new burger:
term)
   TemperatureSensor
brick:Temperature_Sensor
   cookedOn           (none: a new burger:
term)
   hasCalories        (none: a new burger:
term)
   hasIngredient      (none: a new burger:
term)
```

```
   hasLocation        brick:hasLocation
   hasPoint           brick:hasPoint
   oilSetpoint        (none: a new burger:
term)
```

Four of the eleven terms already exist in Brick. The other seven (`Fryer`, `MenuItem`, `oilSetpoint`, …) are restaurant words, and Brick, rightly, has no idea what a fryer is. So the burger ontology gets its own namespace (`burger:`) for all eleven, and on Day 40 we say how the four overlap with Brick. This is exactly the pattern Brick recommends for extensions.

How this maps to buildings

> **Buildings box.** For an FDD project, the competency questions are the rule inputs: “which AHUs have a supply air temperature sensor and a fan status?”, “which VAVs does AHU_1 feed?”. Write them first, and you know which **Brick** classes, **ASHRAE 223P** connections or **Haystack** tags your model must have, and you can test the model against them before any rule runs.

Why it matters

Most ontology projects that fail try to model everything. Competency questions keep you to what someone needs, and give you a test for “done”.

Week 8 further reading

- N. F. Noy and D. L. McGuinness, Ontology Development 101: A Guide to Creating Your First Ontology (Stanford, 2001). The method this week follows.
- METHONTOLOGY (Fernández-López, Gómez-Pérez and Juristo, 1997): a more formal, life-cycle method for building ontologies like software.
- The NeOn methodology (Suárez-Figueroa, Gómez-Pérez and others, around 2012): scenarios for building ontologies by reusing and re-engineering existing ones.
- W3C, OWL 2 Primer, SHACL, SPARQL 1.1 Query
- Brick: extending Brick
- csml2rdf `bacnet.ttl`: a real ontology that uses named enumeration values and classes that are their own SHACL shapes (Days 38–39).

Try it

1. Add a sixth competency question: “Which equipment is in the kitchen?” Which new terms does it need?
2. Move “prices” into scope. Write a competency question for it and list its terms.
3. **Mini challenge (Burger Barn):** write a competency question for “which fryer cooks the onion rings?”, then answer it with SPARQL on `burger.ttl` + `burger_site.ttl`.

Answers

1. None: `Equipment`, `hasLocation` and `Kitchen` are already on the list. A good sign.
2. For example “What does each menu item cost?” needs `MenuItem` (have it) and a new `price` property, plus a decision about currency (QUDT or a plain `xsd:decimal` in one currency).
3. **Fryer_2**: `SELECT ?fryer WHERE { site:OnionRings burger:cookedOn ?fryer }`. (The course’s tests check this answer.)

Day 37 — Steps 4–5: classes, hierarchy and properties

Goal: read the classes and properties of the new Burger ontology, `burger.ttl`, and check that every term the competency questions need is defined, labelled and explained.

Read

Steps 4 and 5 of Ontology Development 101 turn Day 36's term list into definitions. Open `burger.ttl`; every term is written the same way:

```
burger:Fryer a owl:Class ;
    rdfs:subClassOf
burger:CookingEquipment ;
    rdfs:label "Fryer" ;
    skos:definition "A vat of hot oil that
cooks food." ;
    rdfs:isDefinedBy
<https://example.org/burger> .
```

Line	Why
`a owl:Class`	it's a class (step 4)
`rdfs:subClassOf`	where it sits in the hierarchy

Line	Why
`rdfs:label`	what people call it
`skos:definition`	what it means, in one sentence. The most useful line for a reader.
`rdfs:isDefinedBy`	which ontology it belongs to

The textbook's tips for step 4, in short: a subclass is a *kind of* its parent ("a fryer is a kind of cooking equipment"; "a fryer basket is a kind of fryer" fails the test, so it's a part, not a subclass); keep siblings at the same level of detail; and name classes in the singular.

For step 5, each property gets a kind (object or datatype, Day 32), a domain and a range. Remember Day 31: domain and range let a reasoner *infer* types; they don't check anything.

`day37_rdf.py` prints the class tree and the properties, then runs three quality checks: every term from Day 36 is defined, and every term has a label and a definition.

Run

```
uv run python lessons/course.py run 37
```

```
Classes (class, parent):
   CookingEquipment   Equipment
   Cooler             None
   Equipment          None
   Fryer              CookingEquipment
   Grill              CookingEquipment
   Ingredient         None
   Kitchen            Space
   MenuItem           None
   Point              None
   Restaurant         Space
   Space              None
   TemperatureSensor  Point
   WalkInCooler       Equipment

Properties (name, kind, domain, range):
   cookedOn       ObjectProperty     None
CookingEquipment
   hasIngredient  ObjectProperty
MenuItem     Ingredient
   hasLocation    ObjectProperty
Equipment    Space
   hasPoint       ObjectProperty
Equipment    Point
   isPointOf      ObjectProperty     None
None
   servesItem     ObjectProperty
Restaurant   MenuItem
   hasCalories    DatatypeProperty
MenuItem     integer
   oilSetpoint    DatatypeProperty   Fryer
decimal

Terms the competency questions need but
burger.ttl lacks: []
Terms without a label: [] | without a
definition: []
```

Three empty lists: nothing missing. `Cooler` has no parent: it's the deprecated old name for a walk-in cooler (Day 43 explains). `isPointOf` has no domain or range of its own because it's the inverse of `hasPoint`, which has both.

How this maps to buildings

> **Buildings box.** Every **Brick** class has an `rdfs:label` and a `skos:definition` (look one up on ontology.brickschema.org), and **ASHRAE 223P** documents each class the same way. When you define your own terms for a site or a product, write the definition first: it's the line the next engineer reads.

Why it matters

An undocumented term gets used three different ways by three people. One definition per term, checked by a script, prevents that.

Try it

1. Add `burger:Fryer` as a subclass of `burger:Grill` instead. What does the tree say, and why is it wrong?
2. Remove the `skos:definition` from `burger:Grill`. What does the script print?

3. **Mini challenge (Burger Barn):** add a `burger:IceMachine` class (a kind of `burger:Equipment`) with a label and a definition. How many classes are there now, and do the checks still pass?

Answers

1. `Fryer -> Grill`. A fryer isn't a kind of grill; both are kinds of cooking equipment.
2. `without a definition: ['Grill'].`
3. **14 classes**, and all three lists stay empty:

```
burger:IceMachine a owl:Class ;
rdfs:subClassOf burger:Equipment ;
    rdfs:label "Ice machine" ;
skos:definition "A machine that makes
ice." .
```

   (The course's tests check this answer.)

Day 38 — Steps 6–7: restrictions and instances

Goal: add restrictions to classes, describe one restaurant with the ontology, keep the two in separate files, and model a fixed list of values (an *enumeration*) as named things instead of text.

Read

Step 6: restrictions. A restriction says what every member of a class has. `burger.ttl` has two:

Restriction	Reads	OWL words
on `Fryer`	every fryer has **some** temperature point	`owl:someValuesFrom burger:TemperatureSensor`
on `MenuItem`	every menu item has **exactly one** calorie number	`owl:cardinality 1`

Others you'll see: `owl:allValuesFrom` (*only* this kind), `owl:minCardinality` / `owl:maxCardinality`. Remember Day 33: under the open world, OWL uses these to *infer*, never to report something missing. A menu

item with no calories doesn't break the restriction to OWL. The shapes on Day 39 check that.

Step 7: instances. `burger_site.ttl` describes one restaurant: two fryers, a grill, a walk-in cooler, three sensors and four menu items. Note its *own* namespace, `site:`. The ontology (`burger:`) is shared by every restaurant; each restaurant's facts live in their own file under their own names. That's the TBox / ABox split from Day 31, made into two files and two namespaces.

Enumerations: named things, not text. How is each menu item cooked? You could write `burger:cookMethod "fried"`. Text is easy to get wrong: `"fryed"`, `"Fried"`, `"deep-fried"` are all different strings, and nothing says which are allowed. Instead, give each allowed value its own name. That's an enumeration, and it lives in `burger_cook_methods.ttl`:

```
burger:CookMethodEnumerationValue a
owl:Class ; rdfs:subClassOf
burger:EnumerationValue .

burger:CookMethod.fried a
burger:CookMethodEnumerationValue ;
    rdfs:label "fried" ;
    burger:name "fried" ;
    burger:value
"0"^^xsd:nonNegativeInteger ;
    skos:definition "Cooked in hot oil." ;
    burger:needsEquipmentClass
burger:Fryer .

burger:cookMethod rdfs:range
burger:CookMethodEnumerationValue .   # an
IRI, not xsd:string
```

Now an item points at a *thing*: `site:Fries burger:cookMethod burger:CookMethod.fried`. A typo in an IRI is a different IRI, which a shape (tomorrow) catches. And a named value can carry facts a string can't: a definition, a number code for the till, the equipment it needs. (The dot in `CookMethod.fried` is just part of the name.)

Keep names clean. The name (`"fried"`) and the code (`0`) are separate properties. Don't pack them into one string like `"0-fried"` or a label like `"15-Fried pickles"`: then every query has to cut the string apart, and the code can't be checked as a number.

Same code, different meaning. Each franchise owner numbers their own specials, so "special 7" is a *Moo melt* at franchise 101 and a *Spicy stack* at franchise 202 (`burger_specials.ttl`). The code alone is ambiguous; it resolves only with the franchise ID. The fix is the same idea again: one named thing per (franchise, code) pair.

These three patterns come from a real ontology: the BACnet ontology generated by csml2rdf. Every BACnet enumeration value there is a named thing (`bacnet:AbortReason.other` with `bacnet:name "other"` and `bacnet:value 0`), and BACnet's vendor-specific (proprietary) value ranges only mean something together with the vendor's identifier.

`day38_rdf.py` lists the restrictions, counts the instances per class, and checks that the instances only use classes the ontology defines. Then it lists the cook methods, shows each menu item's (finding the one written as text), finds a label with a code packed in, and resolves "special 7" with and without the franchise.

Run

```
uv run python lessons/course.py run 38
```

```
Restrictions (class, property, rule):
   Fryer hasPoint some TemperatureSensor
   MenuItem hasCalories exactly 1

TBox: 125 triples | ABox: 52 triples
Instances per class: {'Fryer': 2, 'Grill':
1, 'Ingredient': 2, 'Kitchen': 1,
'MenuItem': 4, 'Restaurant': 1,
'TemperatureSensor': 3, 'WalkInCooler': 1}
Classes used in the ABox but not defined
in the TBox: []

Cook methods (named value, name, code):
[('CookMethod.fried', 'fried', 0),
('CookMethod.grilled', 'grilled', 1),
('CookMethod.none', 'none', 2)]
Cook method of each menu item:
{'ClassicBurger': 'grilled', 'Cola':
'none', 'CrispyChicken': 'grilled',
'FriedPickles': 'fried', 'Fries': 'fried',
'OnionRings': 'fried', 'Shake': "TEXT
'fried'"}
Cook methods written as text instead of a
named value: ['Shake']
Labels with a code packed in: ['15-Fried
pickles']

What does 'special 7' mean on its own?
['Moo melt', 'Spicy stack']
With the franchise ID, each receipt
resolves: {'Receipt_1': 'Moo melt',
'Receipt_2': 'Spicy stack'}
```

How this maps to buildings

> **Buildings box. Brick** is one TBox shared by thousands of building models (ABoxes), each in its own namespace. **ASHRAE 223P** expresses most "every X has a Y" rules as SHACL shapes rather than OWL restrictions, because a building model needs them *checked*, not inferred. **Haystack** works the same way at heart: shared defs (the TBox) and per-site records (the ABox).
>
> **Enumerations in building ontologies. ASHRAE 223P** does exactly today's pattern: `s223:EnumerationKind` is a class tree of named values (`s223:EnumerationKind-DayOfWeek`, media such as `s223:Fluid-Air`), never free text. **Brick** uses named things for units (QUDT's `unit:DEG_F`, not the text "°F") but plain strings checked with `sh:in` for some value lists. **Haystack** enum tags are strings; the tag's def lists the allowed ones. **BACnet** itself is full of enumerations (object types, units, event states), and vendor-proprietary values need the vendor ID to mean anything, just like franchise specials.

Why it matters

Keeping the ontology and the data apart means you can publish one and share it, and load any restaurant (or building) against it. Named enumeration values do the same for lists of choices: one agreed set of names that every restaurant's data points at, instead of each kitchen typing its own spelling.

Try it

1. Add a `burger:MenuItem` called `site:Milkshake` with no calories. Does the script complain? Will Day 39's SHACL?
2. Write a restriction "every kitchen is part of some restaurant" (you'd need a `burger:partOf` property first).
3. Which menu items does `site:Restaurant_1` serve? Answer with SPARQL.
4. Add a fourth cook method, `burger:CookMethod.baked`, code 3. Which properties does it need?
5. **Mini challenge (Burger Barn):** write a SHACL shape that allows only the three named cook methods on every menu item (exactly one each), then run it on `burger_cook_data.ttl` plus `burger_cook_mistakes.ttl` (and `burger.ttl`, so fried items count as menu items). Which item fails, and why?

Answers

1. This script doesn't: open world, and the class is defined. Day 39's SHACL does: "A menu item needs exactly one whole-number calorie count."

2. `burger:Kitchen rdfs:subClassOf [ a owl:Restriction ; owl:onProperty burger:partOf ; owl:someValuesFrom burger:Restaurant ] .`

3. **ClassicBurger, Cola, Fries, OnionRings**: `SELECT ?item WHERE { site:Restaurant_1 burger:servesItem ?item . ?item a burger:MenuItem }.`

4. `a burger:CookMethodEnumerationValue`, a clean `rdfs:label` and `burger:name "baked"`, `burger:value "3"^^xsd:nonNegativeInteger` and a `skos:definition`. Add it to the shape's `sh:in` list too.

5. **Shake**: its cook method is the text `"fried"`, and text is not one of the named values.

```
burger:MenuItemCookShape a
sh:NodeShape ;
    sh:targetClass burger:MenuItem ;
    sh:property [
        sh:path burger:cookMethod ;
        sh:in (
burger:CookMethod.fried
burger:CookMethod.grilled
burger:CookMethod.none ) ;
        sh:minCount 1 ;
        sh:maxCount 1 ;
    ] .
```

`sh:hasValue burger:CookMethod.fried` would pin *one* value instead of a list; Day 39 uses it. (The course's tests check this answer.)

Day 39 — Step 8: test the ontology with its competency questions and SHACL

Goal: turn Day 36's competency questions into SPARQL, run them on the restaurant, and validate the data with SHACL rules kept in their own file. Then meet the other way to attach a rule: a class that is its own shape.

Read

The last step of Ontology Development 101 is testing. You have two kinds of test:

1. **Competency questions as SPARQL.** Each Day 36 question becomes a query. If a query can't be written, a term is missing (go back to step 3); if it returns nothing, a fact is missing (step 7). For example, CQ1:

```
SELECT DISTINCT ?item WHERE {
  ?item a burger:MenuItem ;
burger:hasIngredient*/burger:cookedOn
?equipment .
  ?equipment a burger:Fryer .
}
```

2. **SHACL shapes** for the closed-world checks the ontology can't do (Day 33). They live in `burger_shapes.ttl`, **separate from the ontology**: the ontology says what terms *mean* and anyone may reuse it; the shapes say what *this* kind of data file must contain, and different projects can have different shapes for the same ontology. Today's: every fryer has a temperature point and one decimal setpoint, every piece of equipment has a location (and, as a warning, a label), every menu item has one whole-number calorie count.

Two ways to attach a shape. The shapes above name their target: `sh:targetClass burger:Fryer`. There's a second way. A class can *also be* an `sh:NodeShape`. SHACL then checks every instance of that class: an *implicit class target*, no `sh:targetClass` needed. From `burger_cook_methods.ttl`:

```
burger:FriedMenuItem a owl:Class,
rdfs:Class, sh:NodeShape ;
    rdfs:subClassOf burger:MenuItem ;
    sh:property [
        sh:path burger:cookMethod ;
        sh:hasValue
burger:CookMethod.fried ;   # exactly this
named value (Day 38)
        sh:minCount 1 ;
        sh:maxCount 1 ;
    ] .
```

Compare the separate-file version, `burger_cook_shapes.ttl`: a `burger:MenuItemCookShape` with `sh:targetClass burger:MenuItem`, an `sh:in` list of the three cook methods, and a warning for labels that don't start with a letter (Day 38's clean names).

	Separate shapes file + `sh:targetClass`	**Class is its own shape**
Where the rule lives	next to the data project	inside the ontology
Who gets it	whoever loads that shapes file	everyone who loads the ontology
Good for	project rules ("our fryers need a setpoint"); different	rules that are part of what the class *means* ("a fried item is fried")

	Separate shapes file + `sh:targetClass`	**Class is its own shape**
	projects, different rules	

Week 9 recommends separate files for your own project rules, and that still holds. Class-as-shape is the right pick when the rule *is* the definition and should travel with the class. Big ontologies do it this way: most classes in **Brick** 1.4 are `owl:Class, sh:NodeShape`, **ASHRAE 223P** declares its classes `s223:Class, sh:NodeShape`, and the object types in the csml2rdf BACnet ontology are `rdfs:Class, sh:NodeShape`.

The SHACL spec's exact rule: the node must be an `rdfs:Class` *and* an `sh:NodeShape` in the shapes graph. That's why `burger:FriedMenuItem` says `rdfs:Class` as well as `owl:Class` (pySHACL also accepts `owl:Class` on its own, but saying both follows the spec to the letter).

`day39_rdf.py` runs all five questions, lists any without an answer, then validates. Last, it checks the cook-method data (with Day 38's three mistakes) both ways.

Run

```
uv run python lessons/course.py run 39
```

```
CQ1 Which menu items depend on a fryer?
   [('Fries',), ('OnionRings',)]
CQ2 Which equipment has no temperature
sensor?
   [('Cooler_1',)]
CQ3 Where is each piece of equipment?
   [('Cooler_1', 'Kitchen_1'), ('Fryer_1',
'Kitchen_1'), ('Fryer_2', 'Kitchen_1'),
('Grill_1', 'Kitchen_1')]
CQ4 What oil temperature is each fryer set
to?
   [('Fryer_1', '175.0'), ('Fryer_2',
'180.0')]
CQ5 How many calories is each menu item?
   [('ClassicBurger', '540'), ('Cola',
'150'), ('Fries', '320'), ('OnionRings',
'410')]

Unanswered questions: []
Data follows the shapes: True

Cook-method check, separate file:
   FriedPickles Warning Keep labels clean:
start with a letter, no codes like '15-'
in front.
   Shake Violation A menu item needs
exactly one cook method, one of the
CookMethod values.

Cook-method check, class is shape:
   CrispyChicken Violation A fried menu
item has exactly one cook method:
CookMethod.fried.
```

Every question answered, and the data passes the shapes. CQ2 is worth a second look: the walk-in cooler has no temperature sensor. The shapes allow that (only fryers must have one). Whether they *should* is a design decision, and today's mini challenge makes it.

The two cook-method checks find different things. The separate file's `sh:in` list catches **Shake** (text, not a named value) and warns about **FriedPickles**' coded label. The class-as-shape catches **CrispyChicken**: "grilled" is a perfectly good cook method, so `sh:in` lets it through, but a *fried* menu item must be fried.

How this maps to buildings

> **Buildings box.** This is how an FDD model gets accepted: run the rule-input questions ("every AHU's supply-air temperature sensor") as SPARQL, run the **Brick** or **ASHRAE 223P** SHACL shapes plus your project's own, and only then switch the rules on. **Haystack** projects do the same with tag queries and their validation tools.
>
> **Brick** and **223P** ship their rules as class-as-shape: load the ontology and `brick:Air_Handling_Unit` (an `owl:Class` *and* an `sh:NodeShape`) already checks its own instances. Your project's extra

> rules (this site's AHUs need a discharge-air sensor) go in a separate shapes file with `sh:targetClass`.

Why it matters

"Done" for an ontology means: every competency question has a query that answers it, and real data passes the shapes. Both are scripts, so you can re-run them on every change.

Try it

1. Delete `burger:oilSetpoint 180.0` from `site:Fryer_2`. What changes in CQ4, and in the SHACL result?
2. Add a sixth competency question, "Which kitchen is each fryer in?", and its query.
3. Delete `owl:Class, rdfs:Class` from `burger:FriedMenuItem` (keep `sh:NodeShape`). Does CrispyChicken still fail?
4. **Mini challenge (Burger Barn):** write a shape that requires every walk-in cooler to have a point. Which cooler breaks it?

Answers

1. CQ4 loses Fryer_2, and SHACL fails: "A fryer needs exactly one decimal oil setpoint."

2. `SELECT ?fryer ?kitchen WHERE { ?fryer a burger:Fryer ; burger:hasLocation ?kitchen . ?kitchen a burger:Kitchen }`

3. No. It's still a shape, but no longer a class, so it has no implicit target and checks nothing. (pySHACL is lenient the other way: a class with `sh:property` but no `sh:NodeShape` still gets checked. The spec asks for both, so declare both and every SHACL tool agrees.) (The course's tests check this.)

4. **Cooler_1**:

```
burger:CoolerShape a sh:NodeShape ;
    sh:targetClass burger:WalkInCooler
;
    sh:property [ sh:path
burger:hasPoint ; sh:minCount 1 ] .
```

   (The course's tests check this answer.)

Day 40 — Reuse, part two: aligning your terms with Brick

Goal: link the burger terms to Brick with the right strength of link, so a Brick-only query finds burger equipment too, and know when *not* to link.

Read

Day 36 found four burger terms that Brick already has. Instead of deleting ours, we **align** them: a separate file, `burger_alignment.ttl`, says how each relates to Brick. Choose the **weakest link that is true**:

Link	Means	Logic for a reasoner	Burger example
`rdfs:subClassOf`	every X is a Y (not the other way)	yes	`burger:Equipment rdfs:subClassOf brick:Equipment`
`rdfs:subPropertyOf`	every use of p is a	yes	`burger:hasPoint`

Link	Means	Logic for a reasoner	Burger example
	use of q		`rdfs:subPropertyOf brick:hasPoint`
`owl:equivalentClass`	X and Y mean exactly the same	yes, merges members	`burger:TemperatureSensor` and `brick:Temperature_Sensor`
`skos:exactMatch`	same meaning, said for people and search tools	none	the same pair
`skos:closeMatch`	similar, **not** the same	none	`burger:Restaurant` and `brick:Building`

Why not make a restaurant `equivalentClass` building? Because a food-court burger stand is a restaurant but not a building. Claim too much and every reasoner downstream draws wrong conclusions. `skos:closeMatch` points readers to the similar term and lets no software conclude anything.

This is exactly how **Brick's own guide to extensions** (Extending Brick) says to do it: put your terms in your own namespace, give each `a owl:Class`, a `skos:definition` and an `rdfs:subClassOf` link to the nearest Brick class, and use `rdfs:subPropertyOf` for relationships that specialize Brick's. (Its example extension even has an ice machine.)

`day40_rdf.py` loads the tiny Brick building, the burger ontology, the restaurant and the alignment, then asks four **Brick-only** questions, with no reasoner and then with one.

Run

```
uv run python lessons/course.py run 40
```

```
brick:Equipment
   no reasoner: ['AHU_1', 'Cooler_1',
'Fryer_1', 'Fryer_2', 'Grill_1', 'VAV_1',
'VAV_2']
   reasoner:    ['AHU_1', 'Cooler_1',
'Fryer_1', 'Fryer_2', 'Grill_1', 'VAV_1',
'VAV_2']
brick:Temperature_Sensor
   no reasoner: ['AHU_1_SAT',
'VAV_1_ZoneTemp']
   reasoner:    ['AHU_1_SAT',
'Fryer_1_OilTemp', 'Fryer_2_OilTemp',
'Grill_1_SurfaceTemp', 'VAV_1_ZoneTemp']
brick:hasPoint
   no reasoner: ['AHU_1', 'Fryer_1',
'Fryer_2', 'Grill_1', 'VAV_1', 'VAV_2']
   reasoner:    ['AHU_1', 'Fryer_1',
'Fryer_2', 'Grill_1', 'VAV_1', 'VAV_2']
brick:Building
   no reasoner: ['Building_1']
   reasoner:    ['Building_1']
```

One query, `?x a/rdfs:subClassOf* brick:Equipment`, now finds the AHU, the VAVs **and** the fryers, grill and cooler, through `rdfs:subClassOf`. The temperature sensors need the reasoner, because a `subClassOf*` path doesn't follow `owl:equivalentClass`. And `brick:Building` stays just `Building_1`: closeMatch carries no logic, as intended.

How this maps to buildings

> **Buildings box.** This is the everyday job of building ontology work: a vendor's or site's own terms aligned to **Brick** (and often to **ASHRAE 223P** and **Haystack**), so standard queries and FDD rules work on them. A site-specific "Kitchen Exhaust Hood" becomes `rdfs:subClassOf brick:Equipment` (or a more specific Brick class); a Haystack-tagged point gets an `skos:exactMatch` or a class from the Haystack-to-Brick mapping.

Why it matters

Alignment is what makes your ontology useful to tools you didn't write. Strong links you can't defend are worse than none, so pick the weakest true one.

Try it

1. Change `skos:closeMatch` to `owl:equivalentClass` for Restaurant/Building and re-run. What does `brick:Building` return now, and why is that a problem?
2. Which Brick class might `burger:WalkInCooler` sit under? (Look it up

in the Brick docs.)

3. **Mini challenge (Burger Barn):** delete the `owl:equivalentClass` line (keep the `skos:exactMatch`) and run the reasoner. Which `brick:Temperature_Sensor`s are left?

Answers

1. It adds `Restaurant_1`. Now every "for each building" FDD rule thinks the burger restaurant is a building.
2. A good fit is `brick:Equipment` (or a more specific refrigeration class if your Brick version has one). Choose the nearest class that is truly a parent.
3. Only **AHU_1_SAT** and **VAV_1_ZoneTemp**: `skos:exactMatch` carries no logic, so the burger sensors stay out. (The course's tests check this answer.)

Further reading — Week 8

The lessons first, then these for exact behavior.

- Noy & McGuinness: Ontology Development 101
- W3C: OWL 2 Web Ontology Language Primer
- W3C: Shapes Constraint Language (SHACL)
- W3C: SPARQL 1.1 Query Language
- csml2rdf bacnet.ttl: enumerations and class-as-shape in a real ontology
- W3C: SKOS reference (definitions, exactMatch, closeMatch)
- Brick: extending Brick